GORKY: голосовой ассистент задач и встреч, в котором модель выбирает действие, а исполняет код

Родион Музыкант, Анастасия Безусова, Илья Бурмистров, Мария Ворона, Денис Чичеров, Давид Егиазарян
НИУ ВШЭ — Нижний Новгород

https://github.com/anastbezu/gorkycode-test · https://gorky.bond

Аннотация

GORKY — голосовой ассистент, который ведёт личные задачи, напоминания и встречи по свободной речи: создаёт записи, показывает планы за период, находит старые записи по смыслу, переносит и отменяет их, переспрашивает, когда данных не хватает. Ключевое решение — разделение ролей: языковая модель за один вызов на реплику выбирает одно из восьми действий и заполняет его аргументы по строгой схеме, а проверки, поиск цели, запись в базу и текст ответа выполняет код. Поиск устроен в два уровня: «точно» по основам слов и «похоже» по векторам; менять запись можно только по точному совпадению. Качество агента измеряет бенчмарк из 581 кейса (747 реплик) в девяти наборах на настоящей модели и настоящей базе: пройдено 95,9 % кейсов, выбор действия — macro-F1 0,967, медиана времени реплики — 1,12 с. Поиск на 457 запросах с ответом находит нужную запись в 96,5 % случаев и ни разу не выбрал чужую запись для изменения среди 587 целей.

1 Введение

Фраза «перенеси её на пятницу» короткая, но для ассистента в ней несколько задач: понять, что это перенос, а не новая запись; найти, какую запись пользователь называет «её»; вычислить дату ближайшей пятницы от текущего момента в его часовом поясе; сохранить прежнее время; проверить, что новая дата не в прошлом, и предупредить о пересечении с другой встречей. Если всё это поручить языковой модели целиком, ошибки трудно заметить: модель может ответить «Готово», не сохранив запись, изменить похожую, но чужую запись или ошибиться в дате на неделю.

GORKY решает это разделением ответственности:

- **один вызов модели на реплику**: модель возвращает одно действие из восьми по строгой схеме (tool calling), всё остальное делает код; нет многошаговых рассуждений и повторных вызовов — поведение предсказуемо, запросов к модели мало;
- **состояние разговора на сервере**: вместо истории сообщений модель получает короткую выжимку — показанные записи под номерами, запись «в фокусе», черновик;
- **безопасная цель изменения**: код меняет только точно найденную запись, похожую лишь предлагает вопросом, любое изменение откатывается словом «верни»;
- **двухуровневый поиск**: «точно» по основам слов и «похоже» по векторам локальной модели эмбеддингов;
- **бенчмарк**: 581 кейс в девяти наборах, прогон на настоящей модели и базе, метрики по действию, по каждому полю и по времени ответа.

2 Архитектура

Система — веб-приложение из трёх контейнеров: PostgreSQL с расширением pgvector, бэкенд на Python 3.13 и FastAPI, страница на React и TypeScript (рис. 1). Голос распознаётся на сервере моделью Gemini (запись до 30 с), после чего путь запроса не зависит от того, говорил пользователь или печатал. Ответ озвучивается голосом браузера: один текст служит и для экрана, и для речи.

Бэкенд разделён на модули с договорёнными контрактами: `api` — HTTP; `agent` — разговор и вызов модели; `services` — правила записей, которыми пользуются и агент, и кнопки интерфейса без модели; `search` — поиск; `db` — модели SQLAlchemy и миграции Alembic. Модель доступна через OpenAI-совместимый API, поэтому провайдер меняется настройкой. Запись и состояние разговора сохраняются в одной транзакции: ассистент не может «запомнить» изменение, которого нет в базе, и наоборот. Поддержаны гости без регистрации и учётки по логину и паролю; записи учётки можно отправлять в отдельный Google Календарь.

3 Агент

3.1 Один вызов модели на реплику

Модель получает системный промпт с текущим временем, днём недели и календарём на две недели, выжимку состояния разговора и фразу пользователя, и

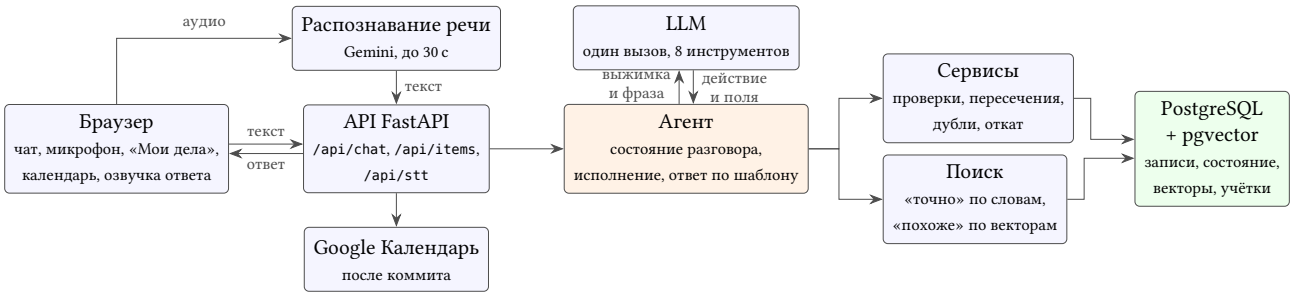


Рис. 1: Архитектура GORKY. Голос превращается в текст на сервере, дальше путь общий для голоса и клавиатуры. Агент вызывает модель один раз на реплику; запись в базу, поиск и текст ответа — код. Запись и состояние разговора сохраняются одним коммитом.

Инструмент	Когда
`create_record`	создать задачу, напоминание, встречу
`list_records`	записи за период: «что у меня завтра»
`search_records`	по теме или человеку: «встреча с Иваном»
`update_record`	перенести, переименовать, сдвинуть
`set_status`	выполнено, отменено, вернуть в активные
`undo_last`	«верни как было»
`ask_user`	не хватает данных — вопрос
`help`	справка или посторонняя тема

Таблица 1: Восемь инструментов агента.

обязана вызвать ровно один инструмент (табл. 1). Аргументы описаны моделями Pydantic, из которых генерируется JSON-схема инструментов; ответ модели проверяется той же схемой. Дальше работает код: исполняет действие, обновляет состояние и собирает ответ по шаблону. Текст «Готово, встреча создана» никогда не пишет модель — только код после успешной записи.

Часть реплик не требует модели вовсе. Отказ посреди уточнения («не надо», «передумал») код распознаёт по списку фраз и сбрасывает черновик; ответ «номер 3» на вопрос «какую?» код разбирает сам.

3.2 Состояние разговора

Вместо истории сообщений на сервере хранится структура: `shown` — записи в том порядке, в каком их назвали пользователю; `focus` — запись, о которой речь («её», «эту»); `pending` и `missing` — черновик создания и чего в нём не хватает; `candidates` — записи для выбора; `last_action` — прежние значения последнего изменения для отката. В промпт попадает короткая текстовая выжимка этой структуры. Так «вторую» превращается в номер из `shown`, «нет, в 16» — в правку записи в фокусе, а ответ «в 15» на вопрос о времени — в дополнение черновика.

3.3 Цель изменения

Для переноса и смены статуса модель не передаёт идентификаторов: она описывает цель — номером на экране, словом «её», текстом с названным временем или фильтром для «отмени всё на завтра». Переводит описание в запись код (функция `find_target`):

- одна точная запись — действие исполняется;
- несколько — код ничего не меняет и спрашивает «какую?» со списком;
- точных нет, есть похожие — вопрос «Точно такой нет. Может, эту?»;
- массовое действие — подтверждение «Отменить 5 созвонов?», не больше 20 записей.

Похожая запись без ответа пользователя не меняется никогда. Отмена сразу исполняется, но удаление мягкое: «верни» восстанавливает прежнее состояние, «нет, не эту, а ту» откатывает действие и применяет его к другой записи.

3.4 Даты и время

Модель возвращает даты в ISO 8601 со смещением пояса пользователя, а промпт задаёт умолчания: «в 3» без «утра» — 15:00 (часы 1–7 считаются дневными), «утром» — 09:00, «вечером» — 18:00; день недели, совпадающий с сегодняшним, — через неделю; время без дня — сегодня; «до конца недели» — срок в воскресенье 23:59. Встреча без времени сохраняется только после вопроса «На какое время?». Код проверяет, что дата не в прошлом и конец позже начала, не создаёт точный дубль и сообщает о пересечении встреч, не блокируя сохранение.

4 Поиск

Поиск отвечает на «когда у меня была встреча с Иваном» и находит цель изменения. Он устроен в два уровня.

«Точно». Название записи и запрос разбираются на слова; каждое слово сравнивается по основе (стеммер Snowball, Porter, 2001), по лемме (pymorphy, Korobov, 2015) и нечётко — с порогом сходства 85 для опечаток. Служебные слова («с», «про», «по») не учитываются; слово типа («встречу», «задачу») совпадает, если оно есть в названии или запись этого типа: «встречу с Севером» находит «Созвон с Севером». Запись подходит, если совпали все значимые слова запроса.

Метрика	Значение
Базовый набор: hit@5	29/30
Расширенный: нашёл подходящую	441/457 (96,5 %)
падежи имён и компаний	69/69
человек + тема, компания + тема	147/147
опечатки	28/37
синонимы и сокращения	16/21
«Точно» на запросах без ответа	0
Цели изменения: чужая запись	0 из 587

Таблица 2: Поиск на стартовом наборе из 1000 записей.

«Похоже». Если точных нет, запрос и названия сравниваются по векторам модели `rubert-mini-frida` (rubert-mini-frida) размерности 312 в pgvector; показываются до 10 записей со сходством не ниже 0,45. Модель работает на сервере — без ключа, сети и лимитов. Результаты упорядочены по близости ко времени: сначала ближайшие будущие, затем прошедшие от новых к старым.

Замер. На стартовом наборе из 1000 записей поиск проверен двумя наборами (табл. 2). Базовый — 30 запросов с ответом и 6 без ответа; расширенный — 619 запросов по группам граничных случаев (падежи, опечатки, сокращения, фильтры) и 587 целей изменения. Главное свойство для безопасности — ни одна из 587 целей не привела к действию над чужой записью. Слабое место — сокращения и сленг («ТЗ», «КП», «презентуха»): их не ловит ни сравнение слов, ни небольшая модель эмбеддингов.

5 Оценка агента

5.1 Наборы и протокол

Кейс — фраза пользователя или диалог из 2–4 реплик с ожиданием на каждой реплике: какой инструмент вызван, значения его полей (тип, название, начало, срок, период, запрос, цель изменения) и исход (исполнено, ждёт ответа, ждёт выбора). Проверяются только заданные в ожидании поля; названия и запросы — по основам слов, даты — до минуты в поясе пользователя. «Сейчас» во всех кейсах — среда 21.10.2026, 10:00, Europe/Moscow.

Набор `test-cases` составлен командой и включает все примеры из задания. Восемь наборов по действиям сгенерированы моделью Gemini 3.8 Flash (high reasoning) по контрактам модулей: ожидание — только то, что однозначно следует из контракта, без чтения кода агента. Агент в замере работает на другой модели — `gemini-3.5-flash-lite`, поэтому модель не проверяет сама себя.

Прогон идёт через ту же функцию обработки реплики, что и API, с настоящей моделью и PostgreSQL с настоящим поиском; каждый кейс исполняется в своей транзакции с откатом.

Набор	Что проверяет	Кейсов	Пройдено
test-cases	задание и сценарии	176	98,3 %
create	создание	75	98,7 %
list	список за период	50	96,0 %
search	поиск	50	100 %
update	изменение	60	95,0 %
status	статус, массовые	50	88,0 %
dialog	уточнение, выбор	50	92,0 %
traps	подвохи выбора	50	92,0 %
undo	откат	20	95,0 %
Всего		581	95,9 %

Таблица 3: Доля пройденных кейсов по наборам; 747 реплик.

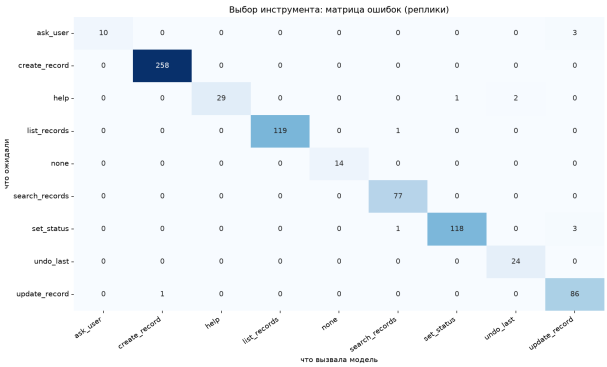


Рис. 2: Матрица ошибок выбора инструмента по 747 репликам: строки — что ожидали, столбцы — что вызвала модель. `none` — ответ кода без модели (отказ).

5.2 Метрики

Кейс пройден, если верны все его реплики. Выбор инструмента оценивается по матрице ошибок: precision, recall и F1 по каждому инструменту и macro-F1 — среднее F1. Для полей считается доля верных значений; если инструмент выбран неверно, все поля реплики засчитываются неверными. Время реплики — от текста пользователя до готового ответа, включая исполнение на базе.

5.3 Результаты

Агент прошёл 557 из 581 кейса (95,9 %, табл. 3). Выбор инструмента — macro-F1 0,967 (рис. 2): точнее всего создание (F1 0,998) и список (0,996), слабее всего вопрос `ask_user` (recall 0,77) — иногда агент действует там, где следовало переспросить. Название записи верно в 199 из 199 реплик, тип — в 218 из 218, начало с признаком «время названо» — в 97,3 %. Медиана времени реплики — 1,12 с, 95-й процентиль — 1,81 с.

5.4 Разбор ошибок

Все 24 непройденных кейса разобраны вручную:

- **не тот инструмент — 9**: изменение вместо смены статуса в «нет, не с Ярославом, а с Ростиславом», поиск вместо отметки «готово, смету отправил»,

правка вместо вопроса, когда ответ «в 16» пришёл после смены темы;
- **даты и время — 7**: «в среду», сказанное в среду, — сегодня вместо следующей недели; «в восемь» — 20:00 вместо 08:00; начало у задачи, где назван только срок;
- **отрицание — 3**: «не отменяй встречу с Иваном» агент исполняет как откат;
- **две команды в фразе — 2**, **цель и статус — 2**;
- **поиск — 1**: «закрой задачу про печать» не находит «Распечатать договор»: однокоренные слова с приставкой не совпадают по основе.

Двадцать три ошибки из двадцати четырёх — ошибки понимания модели; их направление понятно: отрицание и даты закрываются вычислением дат в коде (модель отдаёт сказанное, «пятница, 15:00», а дату считает функция) и отдельным действием «отказ».

6 Ограничения

- Один прогон на одной модели: разброс ответов между прогонами (pass^k, Yao et al., 2024) и другие модели не измерены.
- Восемь из девяти наборов сгенерированы моделью по контрактам: в спорных формулировках эталон следует контракту и может расходиться с ожиданием живого пользователя.
- Проверяются вызов, поля и исход действия, а не формулировка ответа; голосовой тракт (распознавание и озвучка) в замер не входит.
- Агент исполняет только первую команду из фразы, кроме нескольких созданий; повторяющихся записей нет.
- Поиск не знает сокращений и сленга без словаря синонимов.

7 Заключение

Разделение «модель выбирает действие, код исполняет» делает поведение голосового ассистента проверяемым: каждое действие — типизированный вызов, который можно сравнить с эталоном поле за полем. На 581 кейсе агент выбирает верное действие с macro-F1 0,967 и проходит 95,9 % сценариев, а поиск ни разу не выбрал чужую запись для изменения. Наборы, ответы агента на каждый кейс и отчёты лежат в репозитории (`backend/eval/`) и пересобираются одной командой.

Список литературы

Mikhail Korobov. 2015. Morphological analyzer and generator for Russian and Ukrainian languages. In *Analysis of Images, Social Networks and Texts (AIST 2015)*, pages 320–332.

Martin F. Porter. 2001. Snowball: A language for stemming algorithms. https://snowballstem.org.

rubert-mini-frida: компактная модель эмбеддингов для русского языка, дистилляция модели FRIDA. https://huggingface.co/sergeyzh/rubert-mini-frida.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.